SANKHYA — report for the jury

Smart India Hackathon 2026, problem statement SIH26119 (Mangalore Refinery and Petrochemicals Limited): an indigenous GPU-accelerated optimization solver. This report is generated by `tools/make_report.py`. Every number in it is computed from a CSV in `bench/results/` and printed with that file's name and the commit stamped in it; the passages marked "copied as it stands" come verbatim from the document named. CI regenerates it and fails if the committed copy differs.

Contents: 1 What SANKHYA is · 2 Architecture · 3 Headline numbers · 4 Head-to-head · 5 Verification · 6 Honest limits · 7 Provenance · 8 Jury Q&A

1. What SANKHYA is

A mathematical optimization solver written from scratch in C++20: linear programs, mixed integer linear programs, convex quadratic programs, and local nonlinear programs, behind one entry point, with a C API, a command line and Python bindings. It has two families of LP engine: a revised simplex (primal and dual) that produces a basis and is the node solver for branch and cut, and a restarted primal-dual first-order method (PDHG) that runs on the CPU and, through CUDA, on the GPU. An interior point method sits between them.

What sets it apart is not speed. It is that every answer is checked by a program that shares no code with the solver, infeasibility comes with a certificate, optimal values come with a bound that survives floating-point error, and the sensitivity a planner prices from can be re-derived in exact arithmetic. Section 5 gives the numbers and section 6 says where the solver is behind.

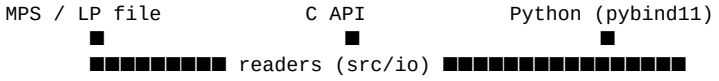
2. Architecture

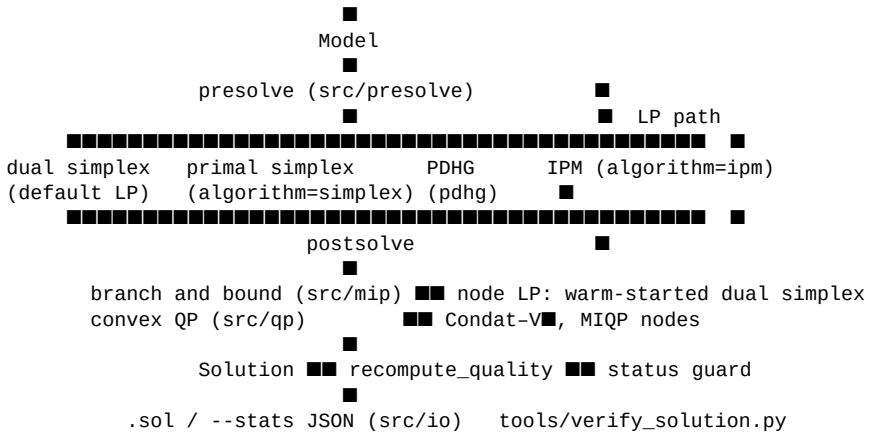
From `docs/ARCHITECTURE.md`, *section "1. The shape in one paragraph", copied as it stands:*

A `Model` goes in, a `Solution` comes out, through one function:

```
solve(const Model&, const Options&) -> Solution        (src/core/solve.cpp)
```

`solve()` classifies the model by what it contains — integrality, a quadratic objective, neither — and dispatches to an engine. Every engine consumes the same `Model`, produces the same `Solution`, and is judged by the same code afterwards: `Solution::recompute_quality()` re-measures feasibility, the reduced costs and the duality gap against the original model, and the dispatcher's status guard downgrades any claim the measurement does not support. That seam is what makes a new engine a bounded piece of work: it has to produce a `Solution`, and it gets the same audit as the ones that exist.





From `docs/ARCHITECTURE.md`*, section "2. Modules and their boundaries", copied as it stands:*

directory	what lives there	depends on	lines			
`include/sankhya/`	the public headers: `model.hpp` (Model, Solution, BasisStatus), `options.hpp`, `tolerances.hpp` (every numerical constant, cited), `sankhya.h` (the C API), `io.hpp`, `mip.hpp`, `qp.hpp`, `pdhg.hpp`	nothing	1.6k			
`src/core/`	`Model`/`Solution` implementation, `recompute_quality()`, the `solve()` dispatcher and its status guard	everything below	0.7k			
`src/util/`	logging, the options registry (every option has a description and a default, checked for duplicates by a test), the version stamp	–	0.8k			
`src/la/`	CSC/CSR sparse matrix with row views, the sparse LU with Markowitz threshold pivoting and the product-form update (FTRAN/BTRAN, eta file), Ruiz + Pock–Chambolle equilibration	–	1.6k			
`src/io/`	MPS (fixed and free, RANGES, negative-UP convention, MARKER blocks, gzip) and LP readers, the `.sol` writer and the `--stats` JSON writer	core	2.2k			
`src/presolve/`	reductions (empty/fixed/singleton rows and columns, redundant rows, free-column singletons, doubleton equations, integer bound rounding) and the postsolve stack that reconstructs the primal and the DUAL of the original model	core, la	1.8k			
`src/simplex/`	`simplex_core.hpp` — the state the two simplex loops share (basis, factors, pricing weights, perturbation, warm start); `primal_simplex.cpp` — bounded-variable revised primal simplex, composite phase 1, Devex pricing, textbook and Harris ratio tests, bound perturbation, basis repair; `dual_simplex.cpp` — bounded dual simplex, bound-flipping ratio test, dual Devex, artificial bounds, cost perturbation, hand-over to the primal loop; `dense_lu` — a dense reference used by tests	core, la	3.2k			
`src/pdhg/`	restarted PDHG (PDLP-style), CPU; the CUDA backend hangs off this path (`src/gpu/`, behind SANKHYA_ENABLE_CUDA, on `main` since #329, measured on an RTX 5050 by #390: the GPU overtakes the CPU at 10,000 rows, `docs/BENCHMARKS.md` section 1g)	core, la	0.7k			
`src/ipm/`	Mehrotra predictor-corrector interior-point method on the normal equations, over the sparse LDL■ in `src/la/ldl.cpp`; no basis	core, la	0.5k			

directory	what lives there	depends on	lines			
`src/qp/`	convexity check (Cholesky of the Hessian), Condat–V■ first-order convex QP	core, la	0.5k			
`src/mip/`	branch and bound: propagation, root diving, reliability branching with strong branching, warm-started dual node LPs, MIQP nodes through the QP engine; root cuts are PR #159	core, simplex, qp	1.3k			
`src/api/`	the C API over `solve()` (including `sankhya_set_callback` and `sankhya_model_interrupt`); the Python bindings (`bindings/python/`, including `Model.interrupt()`) wrap this, not the C++	core	0.5k			
`apps/sankhya-cli/`	`sankhya solve	info	options	version, --stats, --write-sol, --option k=v. Uses sankhya_set_callback` for graceful SIGINT interrupted exit	api, io	–
`tools/`	`verify_solution.py`: re-parses the model with its own reader and checks the `.sol` file's primal feasibility, reduced costs, dual feasibility, complementary slackness and strong duality. Shares no code with the solver, deliberately	–	1.5k			
`tests/`	unit tests per module; `oracles/` — a rational-arithmetic simplex and exact MILP branch and bound that the float engines are fuzzed against; `robustness/` — the sweeps that find where the solver stops working	–	10k			
`bench/runners/`	Netlib, MIPLIB and Mittelmann runners and fetchers, the HiGHS comparison (a separate process over the same files), the robustness sweep, `make_benchmarks_doc.py` which generates `docs/BENCHMARKS.md` from the CSVs	–	4k			

The dependency direction is strictly downward in that table: `la` knows nothing about models, `simplex` knows nothing about integrality, `mip` knows nothing about file formats. No file in `src/` reads or links anything from another optimization solver; `docs/PROVENANCE.md` records the dependency table, the link line and the algorithm citations.

3. Headline numbers, each with its CSV and commit

The file named is the one `docs/BENCHMARKS.md` reads for the same measurement; the commit is the one stamped in the file's rows, the build that produced it.

measurement	CSV	commit
Netlib LP, full set: 94 of 94 within 1e-6 of Koch's exact optimum and verified; 83 match the readme; 94 accepted by the independent verifier	`bench/results/netlib-full-945cb4f.csv`	`945cb4f`
Head-to-head, Netlib LP, 92 instances: SGM10 0.575 s against HiGHS 0.163 s; against HiGHS faster on 3, tie on 72, slower on 17	`bench/results/head-to-head-netlib-5e490ef.csv`	`5e490ef`
Head-to-head, Kennington LP, 16 instances: SGM10 9.430 s against HiGHS 1.855 s; against HiGHS faster on 3, tie on 2, slower on 11	`bench/results/head-to-head-kennington-1860864.csv`	`1860864`
Head-to-head, Maros-Meszaros QP, 138 instances: SGM10 6.437 s against HiGHS 35.781 s; against HiGHS faster on 82, tie on 22, slower on 5	`bench/results/head-to-head-maros-meszaros-1860864.csv`	`1860864`
Safe bounds on Netlib: 91 of 94 optimal answers certified to 1e-6 by a rigorous bound, 2 with no finite bound	`bench/results/certified-gap-netlib-full-945cb4f.csv`	`945cb4f`

measurement	CSV	commit
Netlib infeasible set, verdict with a verified Farkas certificate: 28 of 29	`bench/results/netlib-infeasible-5a39fb2.csv`	`5a39fb2`
Exact sensitivity: 69 of 96 LPs with every dual, reduced cost and range re-derived in rational arithmetic; 4,479 of 431,730 floating-point values corrected	`bench/results/exact-sensitivity-60455ce.csv`	`60455ce`
Kennington LP, matched and verified: 16 of 16	`bench/results/kennington-full-9463fd7.csv`	`9463fd7`
Mittelmann LP, solved and verified inside the limit: 5 of 12	`bench/results/mittelmann-72123ff.csv`	`72123ff`
MIPLIB 2017 easy set: 14 of 30 reach the published optimum, 11 prove it	`bench/results/miplib-ad57c03.csv`	`ad57c03`
Maros-Meszaros convex QP: 117 of 138 at the published objective, 124 verifier-accepted, 0 optimal answers rejected	`bench/results/maros-meszaros-cad720c.csv`	`cad720c`
Hock-Schittkowski NLP: 64 of 70 at the published objective, 70 accepted by the checker	`bench/results/nlp-hs-ad57c03.csv`	`ad57c03`
Convex MINLPLib: 14 of 17 at the published objective, 16 accepted by the checker	`bench/results/nlp-minlplib-ad57c03.csv`	`ad57c03`
Stress set, 126 badly scaled and adversarial LPs: SANKHYA 111 correct / 0 wrong / 15 declined; HiGHS on its defaults 29 / 90 / 7	`bench/results/stress-d32b3847.csv`	`d32b3847`
`sankhya scenarios`: 1130 of 1130 scenarios verified	`bench/results/scenarios-de0f393a.csv`	`de0f393a`
A million rows on the CPU: 2 of 9 engine runs optimal and verified (transport 1,000,000 rows by `pdhg` in 119 s; refinery 1,007,400 rows by `pdhg` in 2,336 s)	`bench/results/million-cpu-b561fba.csv`	`b561fba`
HiGHS agreement, medium Netlib tier: objectives agree on 50 of 50	`bench/results/compare-highs-medium-ad57c03.csv`	`ad57c03`
GPU PDHG on real models, NVIDIA A100-SXM4-40GB: 1.00x to 8.72x against the faster CPU arm over 6 model-tolerance cells, 2 of them ending at the time limit	`bench/results/gpu-real-a100-fdd1f35.csv`	`fdd1f35`

4. Head-to-head against HiGHS, SCIP, CBC/Clp and GLPK

Every solver is a separate process on the same machine, one thread, the same time limit; every answer, a rival's included, goes through `tools/verify_solution.py`, and only a run that is optimal, at the published optimum and verified counts for time. The method is in `docs/BENCHMARKS.md` section 4a, with the performance profiles and every failure named.

Netlib LP

`bench/results/head-to-head-netlib-5e490ef.csv`, commit `5e490ef`, 120 s limit, machine `cloud container (docker); Intel(R) Xeon(R) Processor @ 2.10GHz; 4 cores; 16 GiB RAM; Linux-x86_64`.

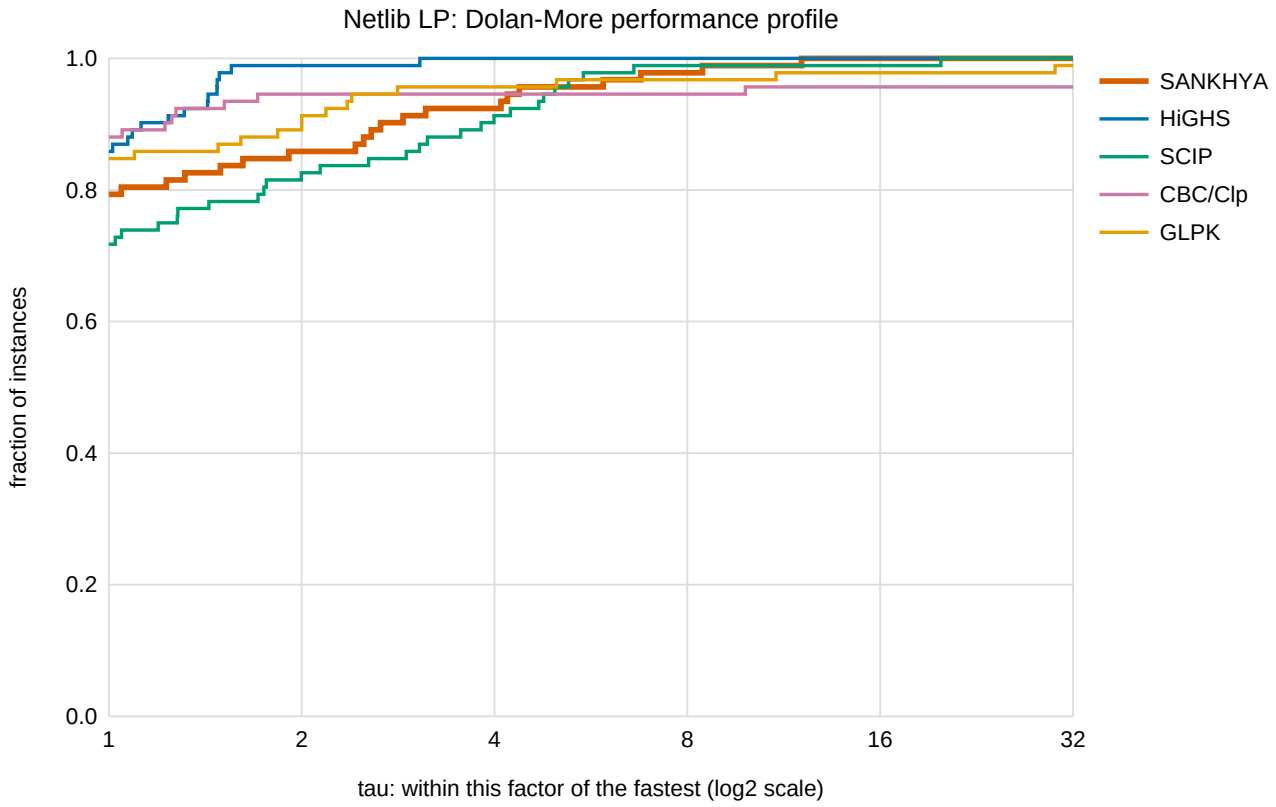
solver	version	runs	solved	matched	matched exact	verified	checked on	SGM time, shift 10 s
SANKHYA	5e490ef	92	92	83	92	92	primal+dual	0.575 s
HiGHS	1.15.1	92	92	83	92	92	primal+dual	0.163 s
SCIP	10.0.2	92	92	83	92	92	primal-only	0.398 s
CBC/Clp	1.17.9	92	92	83	92	88	primal+dual	1.355 s
GLPK	5.0	92	92	83	92	91	primal+dual	0.486 s

Instance by instance, SANKHYA against each rival:

against	instances compared	SANKHYA faster	tie	rival faster	median of SANKHYA's time over the rival's, floored	the same, clocks as recorded
HiGHS	92	3	72	17	1.00x	1.08x

against	instances compared	SANKHYA faster	tie	rival faster	median of SANKHYA's time over the rival's, floored	the same, clocks as recorded
SCIP	92	20	65	7	1.00x	0.42x
CBC/Clp	92	6	71	15	1.00x	0.72x
GLPK	92	7	72	13	1.00x	1.86x

A tie is two times within 10% of each other, or both under 0.1 s (the coarsest clock here cannot tell those apart). A run that did not count loses to one that did, and an instance on which neither counted is left out. Both medians are over the instances both solvers counted on, and above 1x SANKHYA is the slower one: the first floors each time at 0.1 s as the profile below does, the second takes each solver's clock as recorded, which on instances of a few milliseconds is a ratio of start-up costs and, for a solver that prints 0.0, is left out.



Dolan-More performance profile, Netlib LP: a curve's height at tau is the fraction of instances on which that solver's run counted and took at most tau times the fastest counted run (`docs/img/profile-netlib.svg`, drawn from the CSV by `bench/runners/perf_profile.py`)

Kennington LP

`bench/results/head-to-head-kennington-1860864.csv`, commit `1860864`, 120 s limit,

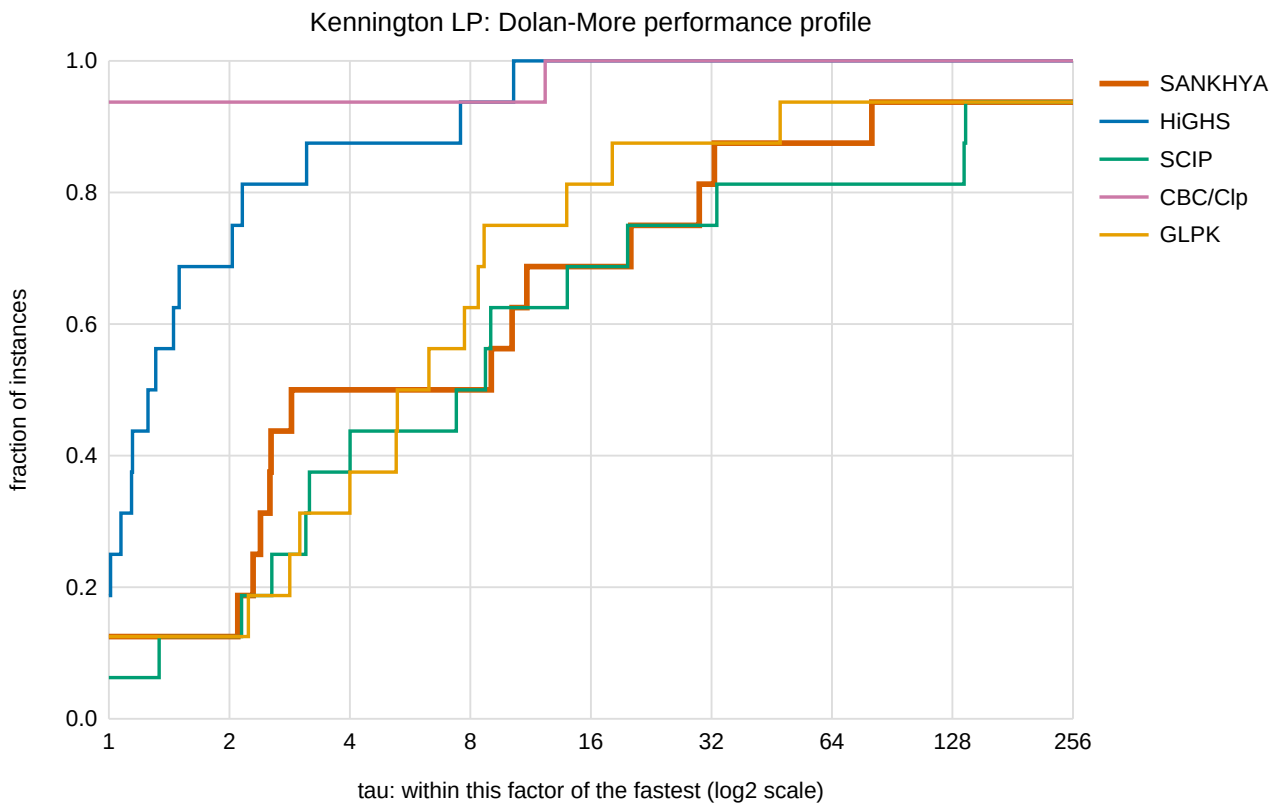
machine `cloud container (docker); Intel(R) Xeon(R) Processor @ 2.10GHz; 4 cores; 16 GiB RAM; Linux-x86_64`.

solver	version	runs	solved	matched	verified	checked on	SGM time, shift 10 s
SANKHYA	1860864	16	15	15	15	primal+dual	9.430 s
HiGHS	1.15.1	16	16	16	16	primal+dual	1.855 s
SCIP	10.0.2	16	15	15	15	primal-only	10.899 s
CBC/Clp	1.17.9	16	16	16	16	primal+dual	0.795 s
GLPK	5.0	16	15	15	15	primal+dual	7.345 s

Instance by instance, SANKHYA against each rival:

against	instances compared	SANKHYA faster	tie	rival faster	median of SANKHYA's time over the rival's, floored	the same, clocks as recorded
HiGHS	16	3	2	11	2.83x	3.12x
SCIP	16	6	1	9	1.25x	1.25x
CBC/Clp	16	1	2	13	2.86x	4.39x
GLPK	16	9	1	6	0.98x	0.74x

A tie is two times within 10% of each other, or both under 0.1 s (the coarsest clock here cannot tell those apart). A run that did not count loses to one that did, and an instance on which neither counted is left out. Both medians are over the instances both solvers counted on, and above 1x SANKHYA is the slower one: the first floors each time at 0.1 s as the profile below does, the second takes each solver's clock as recorded, which on instances of a few milliseconds is a ratio of start-up costs and, for a solver that prints 0.0, is left out.



Dolan-More performance profile, Kennington LP (`docs/img/profile-kennington.svg`)

Maros-Meszaros QP

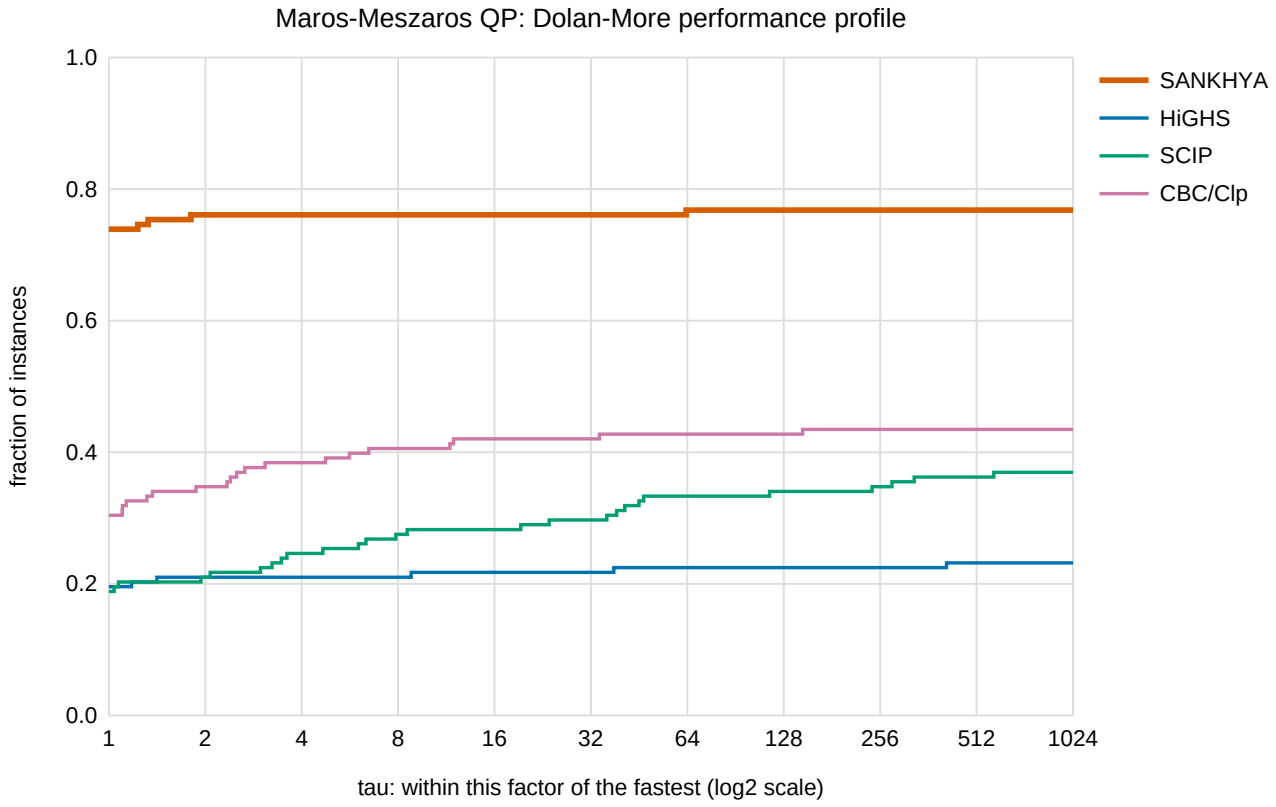
`bench/results/head-to-head-maros-meszaros-1860864.csv`, commit `1860864`, 60 s limit, machine `cloud container (docker); Intel(R) Xeon(R) Processor @ 2.10GHz; 4 cores; 16 GiB RAM; Linux-x86_64`.

solver	version	runs	solved	matched	verified	checked on	SGM time, shift 10 s
SANKHYA	1860864	138	107	106	107	primal+dual	6.437 s
HiGHS	1.15.1	138	105	99	32	primal+dual	35.781 s
SCIP	10.0.2	138	78	76	53	primal-only	27.436 s
CBC/Clp	1.17.9	138	127	113	62	primal+dual	20.974 s
GLPK	-	138	unsupported	-	-	-	-

Instance by instance, SANKHYA against each rival:

against	instances compared	SANKHYA faster	tie	rival faster	median of SANKHYA's time over the rival's, floored	the same, clocks as recorded
HiGHS	109	82	22	5	1.00x	0.26x
SCIP	114	87	19	8	0.52x	0.03x
CBC/Clp	118	75	28	15	1.00x	0.24x

A tie is two times within 10% of each other, or both under 0.1 s (the coarsest clock here cannot tell those apart). A run that did not count loses to one that did, and an instance on which neither counted is left out. Both medians are over the instances both solvers counted on, and above 1x SANKHYA is the slower one: the first floors each time at 0.1 s as the profile below does, the second takes each solver's clock as recorded, which on instances of a few milliseconds is a ratio of start-up costs and, for a solver that prints 0.0, is left out.



Dolan-More performance profile, Maros-Meszaros QP (`docs/img/profile-maros-meszaros.svg`)

5. Verification: why an answer can be believed

- **An independent verifier.** `tools/verify_solution.py` reads only the model file and the written `.sol` file, with its own parsers, and never links the C++. On the full Netlib set it accepts 94 of 94 answers, and 94 of 94 are within 1e-6 of Koch's exact rational optimum (`bench/results/netlib-full-945cb4f.csv`, `docs/BENCHMARKS.md` section 1c). In the head-to-head run the same verifier rejected 5 rival answers on Netlib (`bench/results/head-to-head-netlib-5e490ef.csv`, section 4a.1).
- **Certificates of infeasibility.** 28 of 29 of Netlib's infeasible LPs end with a Farkas certificate the verifier accepts (`bench/results/netlib-infeasible-5a39fb2.csv`,

section 3a). A verdict with no certificate is not counted.

- **Safe bounds.** 91 of 94 optimal Netlib answers carry a

Neumaier-Shcherbina bound within 1e-6 of the objective, re-derived exactly by the

verifier; 2 have no finite bound and say so (`bench/results/certified-gap-netlib-full-945cb4f.csv`,

section 1c.2).

- **Exact sensitivity.** On 69 of 96 LPs every dual, reduced cost and range

was re-derived in rational arithmetic, correcting 4,479 of 431,730

floating-point values (`bench/results/exact-sensitivity-60455ce.csv`; `docs/PROVENANCE.md` section 3).

- **Agreement with an established solver.** HiGHS, as a separate process, agrees on the

objective on 50 of 50 medium-tier Netlib instances (`bench/results/compare-highs-medium-ad57c03.csv`,

section 4).

6. Honest limits

- **Speed against HiGHS.** Netlib SGM 0.575 s against

0.163 s; Kennington 9.430 s against

1.855 s, slower on 11 of

16 instances (`bench/results/head-to-head-netlib-5e490ef.csv`,
`bench/results/head-to-head-kennington-1860864.csv`).

- **Large LPs.** Mittelmann: 5 of 12 inside the limit

(`bench/results/mittelmann-72123ff.csv`, `docs/BENCHMARKS.md` section 1d).

- **MILP proofs.** 11 of 30 MIPLIB instances proved, 14

at the published optimum (`bench/results/miplib-ad57c03.csv`, section 2).

- **Declined instances.** Stress set: 15 of 126 declined,

0 wrong (`bench/results/stress-d32b3847.csv`, section 5b). Maros-Meszaros: 119

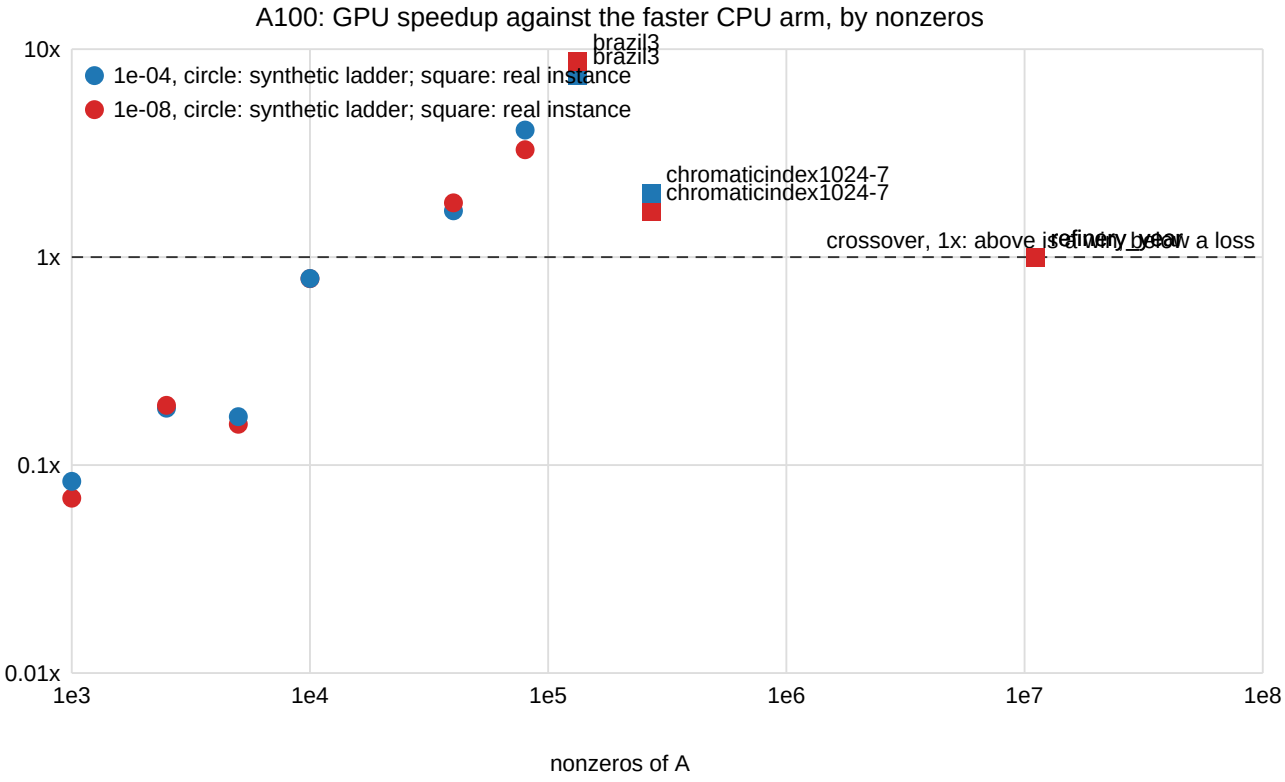
of 138 optimal (`bench/results/maros-meszaros-cad720c.csv`, section 2b). Safe bounds: 2 of

94 with no finite bound (`bench/results/certified-gap-netlib-full-945cb4f.csv`).

- **No real industrial model.** Every refinery number is from a generated model.
- **The GPU, against the faster CPU arm of the same engine** (`bench/results/gpu-real-a100-fdd1f35.csv`, commit `fdd1f35`, NVIDIA A100-SXM4-40GB; `docs/BENCHMARKS.md` section 1g.3, where the small-model losses

of section 1g sit beside it):

model	rows	tolerance	card	faster CPU arm	card status	speed-up
`brazil3`	14,646	1e-04	5.2 s	38.7 s (`cpu-16t`)	feasible	7.50x
`brazil3`	14,646	1e-08	10.5 s	91.8 s (`cpu-16t`)	feasible	8.72x
`chromaticindex1024-7`	67,583	1e-04	0.4 s	0.9 s (`cpu-16t`)	feasible	2.02x
`chromaticindex1024-7`	67,583	1e-08	0.5 s	0.9 s (`cpu-16t`)	feasible	1.65x
`refinery_year`	779,640	1e-04	300.5 s	300.5 s (`cpu-1t`)	time_limit	1.00x
`refinery_year`	779,640	1e-08	300.5 s	300.5 s (`cpu-1t`)	time_limit	1.00x



GPU speed-up against nonzeros on the A100, the synthetic ladder and the real models of `docs/BENCHMARKS.md` section 1g.3 (`docs/img/gpu-speedup-a100.svg`, drawn by `bench/runners/gpu_plot.py`)

From `docs/BENCHMARKS.md`*, section "6. What these numbers do not say", copied as it stands:*

- **The large-model evidence is sections 1d and 1f to 1f.3, and none of it is a real million-variable industrial model.** 1d is Mittelmann's set, a table of named time limits. 1f to 1f.3 are generated instances whose optimum is exact by construction: under a clock the first-order engine reaches 100,000 rows and columns, the interior point 5,000 on the random shape and 20,000 on the staircase, and the dual simplex 1,000; under a fixed iteration budget the random shape goes to 1,000,000; the largest refinery-shaped model solved exactly is 32,485 rows. No pass rate in the Netlib sections above substitutes for any of it. Tracked as #198 and as part of #54.
- Wall-clock times at this size are dominated by process start-up and file reading, so ratios between solvers are not meaningful until the instances get big enough to matter. The comparison in section 4 uses solver-internal time on both sides for that reason.
- The failures in section 1b are real and are not going to be quietly dropped from a later edition of this file. Each one carries the issue tracking it.
- The GPU PDHG backend is measured in section 1g. The interior-point method (`algorithm=ipm`, #56) is opt-in and produces no basis, so it is not the engine behind any Netlib or MIPLIB table above - sections 1f to 1f.3 are the exception, where it appears beside the others: since the AMD ordering (#193) it reaches 5,000 rows on the random shape and 20,000 on the staircase, and solves the 32,485-row refinery year exactly (#206, #211). Its own Netlib run is committed as `netlib-full-*-ipm.csv` and quoted in

`docs/PS26119_COVERAGE.md`, not here, because a run made with a non-default option is a measurement of that option rather than the tier's evidence. `docs/PROVENANCE.md` and issue #54 carry the full accounting.

From `docs/NEGATIVE-RESULTS.md`*, section "4. Withdrawn claims", copied as it stands:*

Numbers this project published and then retracted, kept here by name.

- **"Root plus tree rounds reach 14 and prove 10."** Published from the three-way at `078cb24`. The clean re-run at `0018254` — alone on the machine, apps closed — gave root + tree 12 / 9 and root alone 13 / 10. The `078cb24` tree figure did not reproduce; the default was set on the leg that did. Withdrawn in #459.
- **"1.63× faster at 10,000×10,000."** Published from `gpu-a4f02b1.csv`. The re-run `gpu-fb72ab4.csv` gave 1.37× at the same tolerance. Both are now named wherever the figure is stated, and the repeats runner (#449) replaces the single-run figure. Withdrawn in #435.
- **"0 of 8 on Mittelmann."** Published from `mittelmann-e134aeb.csv` for the default engine. The clean re-run at `0018254` on all three engines gave 2 of 8 under `auto` and 3 of 8 across engines; two of the eight are the interior point running out of memory on a 7.7 GB machine, and two are engine-selection misses (#417). Replaced in #459. The out-of-memory reading of that sentence is withdrawn in the next entry.
- **"The interior point is limited by memory on Mittelmann."** Read from the two out-of-memory rows of the `0018254` re-run on the 7.7 GB laptop. The same eight under `algorithm=ipm` on a 96 GB node with 86 GB free (`mittelmann-ipm-5c7efbc.csv`, #575) finish the same 2 of 8, and none of the six misses is memory: three abandon the ordering in 11–22 s at `ipm_max_ordering_entries = 1e8`, a budget sized for the laptop; two decline at the first factorization's share of the limit; `irish-electricity` is `numerical_error` because its point violates a row by 1.0e-4, which the status guard measures (objective 2454405 against HiGHS's 2546254). Memory was the 7.7 GB box's limit, not the engine's. The ordering and factor budgets are now sized from the machine's physical memory, with the laptop constants as the floor (#581); whether that moves the three abandoned orderings is the next ipm leg on the node, not yet run (#576).

7. Provenance

From `docs/PROVENANCE.md`*, section "1. The red line", copied as it stands:*

No source code from an optimization **solver** is copied, vendored, linked, or read. That list explicitly includes CBC, Clp, HiGHS, SCIP, SoPlex, GLPK, lp_solve, OSQP, PDLP, cuPDLP / cuPDLP-C / cuPDLPx, OR-Tools GLOP and CP-SAT, Gurobi, CPLEX and Xpress — not their simplex, not their cuts, not their MPS readers.

HiGHS appears in this project **only** as the comparison baseline in `bench/runners/compare.py`. It runs either as an externally installed command-line binary or, where none is present, through the `highspy` pip package in a separate Python process. It is never linked into SANKHYA, never a build dependency, and no part of its source informs ours. The comparison results are in `docs/BENCHMARKS.md` section 3.

The dependency table, the algorithm citation table, the link line and the SBOM are in `docs/PROVENANCE.md` sections 2 to 4; `docs/sbom.spdx.json` is the committed bill of materials.

8. Jury Q&A

Twenty-five questions a refinery or optimization judge is likely to ask. Each answer ends

with the file or section it rests on; a number in an answer is the one computed for

section 3.

Q1. Is this built on top of an existing solver?

No. No source of any optimization solver is copied, linked or read, and a CI job dumps what the binary links against and fails if a solver library appears. Rival solvers are run only as separate processes, to compare against.

Evidence: `docs/PROVENANCE.md` *sections 1 (the red line), 2 (dependency table) and 4.1 (what the binary links against).*

Q2. Which algorithms are implemented, and where do they come from?

Revised primal and dual simplex, restarted PDHG on CPU and CUDA, an interior point method, presolve with postsolve, branch and cut, convex QP engines and a local NLP interior point. Each is written from the literature and carries its citation above the implementation.

Evidence: `docs/PROVENANCE.md` *section 3 (algorithm citation table: algorithm, paper, file);* `docs/PS26119_COVERAGE.md`, *"Algorithms the PS names".*

Q3. How do I know an answer is right, not just plausible?

Every solve writes a `.sol` file, and `tools/verify_solution.py`, a separate Python program that shares no code with the solver, re-reads the model and re-derives feasibility, dual feasibility, complementarity and the objective. On the full Netlib set it accepts 94 of 94 answers.

Evidence: `bench/results/netlib-full-945cb4f.csv` *at* `945cb4f`*;* `docs/BENCHMARKS.md` *section 1c.*

Q4. The verifier is yours too. Why should I trust it?

Three independent anchors. The Netlib answers are graded against Koch's optima, computed in exact rational arithmetic: 94 of 94 within 1e-6. The same verifier judges the rivals' answers in the head-to-head run and rejected 5 of theirs on Netlib, each named with the violated condition. And HiGHS, as a separate process, agrees on the objective on 50 of 50 medium-tier instances.

Evidence: `bench/results/netlib-full-945cb4f.csv`*;* `bench/results/head-to-head-netlib-5e490ef.csv` *and* `docs/BENCHMARKS.md` *section 4a.1;* `bench/results/compare-highs-medium-ad57c03.csv` *and section 4.*

Q5. How fast is it against HiGHS on Netlib?

Slower in the aggregate. Shifted geometric mean with a 10 s shift over 92 instances: SANKHYA 0.575 s, HiGHS 0.163 s. Instance by instance, SANKHYA is faster on 3, ties on 72 (within 10%, or both under 0.1 s) and is slower on 17. We do not headline a win count: most pairs are ties, because most Netlib instances finish in milliseconds.

Evidence: `bench/results/head-to-head-netlib-5e490ef.csv` *at* `5e490ef`*;* `docs/BENCHMARKS.md` *section 4a.1.*

Q6. And on larger LPs?

On the Kennington set HiGHS is clearly ahead: SGM 9.430 s against 1.855 s, SANKHYA faster on 3 of 16 and slower on 11, a median ratio of 2.83x. On Mittelmann's LPs SANKHYA finishes 5 of 12 inside the limit. Speed at size is the main open work.

Evidence: `bench/results/head-to-head-kennington-1860864.csv`*, section 4a.2;* `bench/results/mittelmann-72123ff.csv`*, section 1d of* `docs/BENCHMARKS.md`*.*

Q7. How does the QP engine compare?

On the Maros-Meszaros convex QPs in the head-to-head run SANKHYA reports optimal on 107 of 138, at the published objective on 106, and the verifier accepts 106 of those; HiGHS reports optimal on 105, at the published objective on 99, and the verifier accepts 32, the rest failing its 1e-7 dual feasibility or 1e-6 gap conditions by small margins (each named in section 4a.3), so the SGM of 6.437 s against 35.781 s rests on the verifier's tolerances as

much as on speed. In SANKHYA's own run 117 of 138 reach the published objective and 0 `optimal` answers are rejected by the verifier.

Evidence: `bench/results/head-to-head-maros-meszaros-1860864.csv`, `docs/BENCHMARKS.md` *section 4a.3;* `bench/results/maros-meszaros-cad720c.csv`, *section 2b.*

Q8. What does the GPU actually buy?

On real models on an NVIDIA A100-SXM4-40GB, the CUDA first-order engine is 1.00x to 8.72x the speed of the faster CPU arm of the same engine on the same host, over 6 model-tolerance cells of which the card ends 2 at the time limit (section 6 prints every cell). It is a speed-up of our own first-order method, measured against our own CPU code; it is not a claim to beat a simplex or barrier solver on the same model.

Evidence: `bench/results/gpu-real-a100-fdd1f35.csv` *at* `fdd1f35`*;* `docs/BENCHMARKS.md` *sections 1g and 1g.3.*

Q9. Where does the GPU lose?

On small models, where launch and transfer costs are not amortised; on root domain propagation and batched node bounds for MILP, which lose to the CPU and stay off; and on the largest refinery year, where the card is further along but 2 of its 6 runs still end at the time limit.

Evidence: `bench/results/gpu-real-a100-fdd1f35.csv`*;* `docs/BENCHMARKS.md` *sections 1g, 1g.5 and 1g.10;* `docs/NEGATIVE-RESULTS.md` *section 1, "The GPU crossover as a single-run figure".*

Q10. Does it reach a million variables?

On generated models with an optimum exact by construction, on the CPU: 2 of 9 engine runs end optimal and verified (transport 1,000,000 rows by `pdhg` in 119 s; refinery 1,007,400 rows by `pdhg` in 2,336 s). The runs that do not finish are in the same table with the reason. The GPU arms at this size have not been run.

Evidence: `bench/results/million-cpu-b561fba.csv` *at* `b561fba`*;* `docs/BENCHMARKS.md` *section 1f.5.*

Q11. Has it solved a real refinery model?

No real industrial model has been made available to us. The refinery evidence is a generated multi-period planning model (crudes, tanks, yields, capacities, quality budgets, deliveries) whose optimum is known by construction, plus scenario sweeps on it. We say so wherever the number appears.

Evidence: `docs/case_studies/refinery.md`*;* `docs/BENCHMARKS.md` *sections 1f.3 and 6.*

Q12. How strong is the MILP side?

Behind mature solvers, and reported that way. On the MIPLIB 2017 easy instances we run, 14 of 30 reach the published optimum and 11 also prove it. The search finds good incumbents more often than it closes the bound.

Evidence: `bench/results/miplib-ad57c03.csv` *at* `ad57c03`*;* `docs/BENCHMARKS.md` *section 2.*

Q13. What do I get when a model is infeasible?

A Farkas certificate, written to the solution file and checked by the verifier: a verdict without a certificate is not counted as a pass. On Netlib's infeasible set 28 of 29 verdicts carry a certificate the verifier accepts.

Evidence: `bench/results/netlib-infeasible-5a39fb2.csv` *at* `5a39fb2`*;* `docs/BENCHMARKS.md` *section 3a.*

Q14. What is a "safe bound", and why does it matter to a planner?

A lower bound on the true optimum computed from the reported duals with outward rounding (Neumaier and Shcherbina), so it holds despite floating-point error, and re-derived exactly by the verifier. Of 94 optimal Netlib answers, 91 are certified within 1e-6 of the true optimum this way, 1 have a looser finite bound and 2 have none, and the file then claims nothing.

Evidence: `bench/results/certified-gap-netlib-full-945cb4f.csv` *at* `945cb4f`*;* `docs/BENCHMARKS.md` *section 1c.2.*

Q15. Can the shadow prices be trusted for pricing decisions?

With the `exact` option the duals, reduced costs and ranges are re-derived from the basis in rational arithmetic and each floating-point value is marked certified or corrected. Over 69 of 96 LPs where the derivation finished in its budget, 4,479 of 431,730 floating-point sensitivity values were corrected. Degenerate rows get a left and a right shadow price instead of one number.

Evidence: `bench/results/exact-sensitivity-60455ce.csv` *at* `60455ce`*;* `docs/PROVENANCE.md` *section 3, "Certified sensitivity".*

Q16. What happens on badly scaled models?

On 126 scaled-Netlib and adversarial LPs SANKHYA is correct on 111, wrong on 0 and declines 15. HiGHS 1.15.1 on its default options is 29 correct, 90 wrong, 7 failed on the same files; its defaults drop the tiny coefficients these files carry, so this measures defaults on extreme scaling, not HiGHS in general.

Evidence: `bench/results/stress-d32b3847.csv` *at* `d32b3847`*;* `docs/BENCHMARKS.md` *section 5b.*

Q17. What does the solver do when it cannot solve a model?

It declines: the status is a limit, `numerical_error` or `feasible`, never `optimal`. The 15 stress-set runs above and the Maros-Meszaros runs short of `optimal` (119 of 138 are optimal) are named instance by instance.

Evidence: `bench/results/stress-d32b3847.csv`*;* `bench/results/maros-meszaros-cad720c.csv`*;* `docs/ARCHITECTURE.md` *section 8 (resource limits and what each one means).*

Q18. Does it handle non-convex QP?

No, and it refuses rather than approximates: convexity is decided by an LDL^T test before any arithmetic, and a non-convex Hessian is returned with the negative pivot as the reason. A local optimum is never reported as a global one.

Evidence: `docs/PS26119_COVERAGE.md`*, "Problem classes";* `docs/BENCHMARKS.md` *section 2d.*

Q19. What about NLP and MINLP?

A local interior point for NLP and branch and bound for MINLP proved convex. Hock-Schittkowski: 64 of 70 at the published objective, 70 accepted by the checker. Convex MINLPLib: 14 of 17. The status says `locally_optimal` unless the model is proved convex.

Evidence: `bench/results/nlp-hs-ad57c03.csv`*;* `bench/results/nlp-minlplib-ad57c03.csv`*;* `docs/BENCHMARKS.md` *section 2e.*

Q20. Can a planner run what-if scenarios?

Yes: `sankhya scenarios` re-solves a model over a set of changes from a warm start, and each scenario's answer is verified; 1130 of 1130 in the committed run. `tools/report.py` prints binding constraints and shadow prices by the model's own names.

Evidence: `bench/results/scenarios-de0f393a.csv` *at* `de0f393a`*;* `README.md`*, "Use";* `docs/FINALE.md`*.*

Q21. How are the benchmark numbers kept honest?

Every run writes a CSV with the instance's sha256, both objectives, the gap, status, time, the git commit and the machine. `docs/BENCHMARKS.md` and this report are generated from those files by scripts, and CI fails if a CSV's commit is not on `main` or if either document is out of date.

Evidence: `docs/BENCHMARKS.md`*, opening section;* `bench/runners/check_result_stamps.py`*;* `tools/make_report.py --check`*.*

Q22. Can I reproduce the results myself?

Yes. `scripts/reproduce.sh` takes a fresh clone through the build, the tests, the Netlib run with independent verification and the HiGHS comparison, offline, and CI runs the same script on a machine that has never built the project.

Evidence: `README.md`, *"Reproduce everything";* `docs/ARCHITECTURE.md` *section 7 (reproducibility, and what is actually promised).*

Q23. What did you try that did not work?

Tree cut rounds, flow cover strengthening, OpenMP in the LP engines and the Forrest-Tomlin update were built, measured and demoted; the single-run GPU crossover figure was withdrawn; and each withdrawn claim is kept by name with the number that withdrew it.

Evidence: `docs/NEGATIVE-RESULTS.md` *sections 1 to 4.*

Q24. How would this be integrated into an existing planning system?

Through a C API shaped like the established solvers' (create, set, solve, query), Python bindings, a CLI, and MPS and LP readers. No GUI is part of the deliverable.

Evidence: `include/sankhya/sankhya.h`*;* `README.md`, *"Use";* `docs/PS26119_COVERAGE.md`, *"Expected solution".*

Q25. How do MIQP, NLP or a new engine get added later?

Every engine implements one entry point, `solve(const Model&, const Options&) -> Solution`, behind a dispatcher; a new engine registers there and gets the same quality audit, status guard and verifier as the existing ones.

Evidence: `docs/ADDING_AN_ENGINE.md`*;* `docs/ARCHITECTURE.md` *sections 5 and 13.*